22 | 阶段实操：构建一个简单的 KV server（1）-基本流程（下）

2021-10-11 陈天

《陈天 · Rust 编程第一课》 课程介绍 >



讲述：陈天
时长 18:34 大小 17.01M

你好，我是陈天。

上篇我们的 KV store 刚开了个头，写好了基本的接口。你是不是摩拳擦掌准备开始写具体实现的代码了？别着急，当定义好接口后，先不忙实现，在撰写更多代码前，我们可以从一个使用者的角度来体验接口如何使用、是否好用，反观设计有哪些地方有待完善。

还是按照上一讲定义接口的顺序来一个一个测试：首先我们来构建协议层。

实现并验证协议层

先创建一个项目：`cargo new kv --lib`。进入到项目目录，在 Cargo.toml 中添加依赖：

复制代码

```
[package]
name = "kv"
version = "0.1.0"
edition = "2018"

[dependencies]
bytes = "1" # 高效处理网络 buffer 的库
prost = "0.8" # 处理 protobuf 的代码
tracing = "0.1" # 日志处理

[dev-dependencies]
anyhow = "1" # 错误处理
async-prost = "0.2.1" # 支持把 protobuf 封装成 TCP frame
futures = "0.3" # 提供 Stream trait
tokio = { version = "1", features = ["rt", "rt-multi-thread", "io-util", "macr
tracing-subscriber = "0.2" # 日志处理

[build-dependencies]
prost-build = "0.8" # 编译 protobuf
```

然后在项目根目录下创建 abi.proto，把上文中 protobuf 的代码放进去。在根目录下，再创建 build.rs：

复制代码

```
fn main() {
    let mut config = prost_build::Config::new();
    config.bytes(&["."]);
    config.type_attribute(".", "#[derive(PartialOrd)]");
    config
        .out_dir("src/pb")
        .compile_protos(&["abi.proto"], &["."])
        .unwrap();
}
```

这个代码在第 5 讲已经见过了，build.rs 在编译期运行来进行额外的处理。

这里我们为编译出来的代码额外添加了一些属性。比如为 protobuf 的 bytes 类型生成 Bytes 而非缺省的 Vec<u8>，为所有类型加入 PartialOrd 派生宏。关于 prost-build 的扩展，你可以看文档。

记得创建 src/pb 目录，否则编不过。现在，在项目根目录下做 `cargo build` 会生成 src/pb/abi.rs 文件，里面包含所有 protobuf 定义的消息的 Rust 数据结构。我们创建 src/pb/mod.rs，引入 abi.rs，并做一些基本的类型转换：

复制代码

```rust
pub mod abi;

use abi::{command_request::RequestData, *};

impl CommandRequest {
    /// 创建 HSET 命令
    pub fn new_hset(table: impl Into<String>, key: impl Into<String>, value: V
        Self {
            request_data: Some(RequestData::Hset(Hset {
                table: table.into(),
                pair: Some(Kvpair::new(key, value)),
            })),
        }
    }
}

impl Kvpair {
    /// 创建一个新的 kv pair
    pub fn new(key: impl Into<String>, value: Value) -> Self {
        Self {
            key: key.into(),
            value: Some(value),
        }
    }
}

/// 从 String 转换成 Value
impl From<String> for Value {
    fn from(s: String) -> Self {
        Self {
            value: Some(value::Value::String(s)),
        }
    }
}

/// 从 &str 转换成 Value
impl From<&str> for Value {
    fn from(s: &str) -> Self {
        Self {
            value: Some(value::Value::String(s.into())),
        }
    }
}
```

最后，在 src/lib.rs 中，引入 pb 模块：

复制代码

```
mod pb;
pub use pb::abi::*;
```

这样，我们就有了能把 KV server 最基本的 protobuf 接口运转起来的代码。

在根目录下创建 examples，这样可以写一些代码测试客户端和服务器之间的协议。我们可以先创建一个 examples/client.rs 文件，写入如下代码：

复制代码

```
use anyhow::Result;
use async_prost::AsyncProstStream;
use futures::prelude::*;
use kv::{CommandRequest, CommandResponse};
use tokio::net::TcpStream;
use tracing::info;

#[tokio::main]
async fn main() -> Result<()> {
    tracing_subscriber::fmt::init();

    let addr = "127.0.0.1:9527";
    // 连接服务器
    let stream = TcpStream::connect(addr).await?;

    // 使用 AsyncProstStream 来处理 TCP Frame
    let mut client =
        AsyncProstStream::<_, CommandResponse, CommandRequest, _>::from(stream

    // 生成一个 HSET 命令
    let cmd = CommandRequest::new_hset("table1", "hello", "world".into());

    // 发送 HSET 命令
    client.send(cmd).await?;
    if let Some(Ok(data)) = client.next().await {
        info!("Got response {:?}", data);
    }

    Ok(())
}
```

这段代码连接服务器的 9527 端口，发送一个 HSET 命令出去，然后等待服务器的响应。

同样的，我们创建一个 examples/dummy_server.rs 文件，写入代码：

复制代码

```rust
use anyhow::Result;
use async_prost::AsyncProstStream;
use futures::prelude::*;
use kv::{CommandRequest, CommandResponse};
use tokio::net::TcpListener;
use tracing::info;

#[tokio::main]
async fn main() -> Result<()> {
    tracing_subscriber::fmt::init();
    let addr = "127.0.0.1:9527";
    let listener = TcpListener::bind(addr).await?;
    info!("Start listening on {}", addr);
    loop {
        let (stream, addr) = listener.accept().await?;
        info!("Client {:?} connected", addr);
        tokio::spawn(async move {
            let mut stream =
                AsyncProstStream::<_, CommandRequest, CommandResponse, _>::fro
            while let Some(Ok(msg)) = stream.next().await {
                info!("Got a new command: {:?}", msg);
                // 创建一个 404 response 返回给客户端
                let mut resp = CommandResponse::default();
                resp.status = 404;
                resp.message = "Not found".to_string();
                stream.send(resp).await.unwrap();
            }
            info!("Client {:?} disconnected", addr);
        });
    }
}
```

在这段代码里，服务器监听 9527 端口，对任何客户端的请求，一律返回 status = 404，message 是 “Not found” 的响应。

如果你对这两段代码中的异步和网络处理半懂不懂，没关系，你先把代码抄下来运行。今天的内容跟网络无关，你重点看处理流程就行。未来会讲到网络和异步处理的。

我们可以打开一个命令行窗口，运行：`RUST_LOG=info cargo run --example dummy_server --quiet`。然后在另一个命令行窗口，运行：`RUST_LOG=info cargo run --example client --quiet`。

此时，服务器和客户端都收到了彼此的请求和响应，协议层看上去运作良好。一旦验证通过，就你可以进入下一步，因为协议层的其它代码都只是工作量而已，在之后需要的时候可以慢慢实现。

实现并验证 Storage trait

接下来构建 Storage trait。

我们上一讲谈到了如何使用嵌套的支持并发的 im-memory HashMap 来实现 storage trait。由于 Arc<RwLock<HashMap<K, V>>> 这样的支持并发的 HashMap 是一个刚需，Rust 生态有很多相关的 crate 支持，这里我们可以使用 dashmap 创建一个 MemTable 结构，来实现 Storage trait。

先创建 src/storage 目录，然后创建 src/storage/mod.rs，把刚才讨论的 trait 代码放进去后，在 src/lib.rs 中引入 “mod storage”。此时会发现一个错误：并未定义 KvError。

所以来定义 KvError。第 18 讲讨论错误处理时简单演示了，如何使用 thiserror 的派生宏来定义错误类型，今天就用它来定义 KvError。创建 src/error.rs，然后填入：

复制代码

```
use crate::Value;
use thiserror::Error;

#[derive(Error, Debug, PartialEq)]
pub enum KvError {
    #[error("Not found for table: {0}, key: {1}")]
    NotFound(String, String),

    #[error("Cannot parse command: `{0}`")]
    InvalidCommand(String),
    #[error("Cannot convert value {:0} to {1}")]
    ConvertError(Value, &'static str),
    #[error("Cannot process command {0} with table: {1}, key: {2}. Error: {}")
    StorageError(&'static str, String, String, String),

```

```
    #[error("Failed to encode protobuf message")]
    EncodeError(#[from] prost::EncodeError),
    #[error("Failed to decode protobuf message")]
    DecodeError(#[from] prost::DecodeError),

    #[error("Internal error: {0}")]
    Internal(String),
}
```

这些 error 的定义其实是在实现过程中逐步添加的，但为了讲解方便，先一次性添加。对于 Storage 的实现，我们只关心 StorageError，其它的 error 定义未来会用到。

同样，在 src/lib.rs 下引入 mod error，现在 src/lib.rs 是这个样子的：

复制代码

```
mod error;
mod pb;
mod storage;

pub use error::KvError;
pub use pb::abi::*;
pub use storage::*;
```

src/storage/mod.rs 是这个样子的：

复制代码

```
use crate::{KvError, Kvpair, Value};

/// 对存储的抽象，我们不关心数据存在哪儿，但需要定义外界如何和存储打交道
pub trait Storage {
    /// 从一个 HashTable 里获取一个 key 的 value
    fn get(&self, table: &str, key: &str) -> Result<Option<Value>, KvError>;
    /// 从一个 HashTable 里设置一个 key 的 value，返回旧的 value
    fn set(&self, table: &str, key: String, value: Value) -> Result<Option<Val
    /// 查看 HashTable 中是否有 key
    fn contains(&self, table: &str, key: &str) -> Result<bool, KvError>;
    /// 从 HashTable 中删除一个 key
    fn del(&self, table: &str, key: &str) -> Result<Option<Value>, KvError>;
    /// 遍历 HashTable，返回所有 kv pair（这个接口不好）
    fn get_all(&self, table: &str) -> Result<Vec<Kvpair>, KvError>;
    /// 遍历 HashTable，返回 kv pair 的 Iterator
    fn get_iter(&self, table: &str) -> Result<Box<dyn Iterator<Item = Kvpair>>
}
```

代码目前没有编译错误，可以在这个文件末尾添加测试代码，尝试使用这些接口了，当然，我们还没有构建 MemTable，但通过 Storage trait 已经大概知道 MemTable 怎么用，所以可以先写段测试体验一下：

复制代码

```rust
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn memtable_basic_interface_should_work() {
        let store = MemTable::new();
        test_basi_interface(store);
    }

    #[test]
    fn memtable_get_all_should_work() {
        let store = MemTable::new();
        test_get_all(store);
    }

    fn test_basi_interface(store: impl Storage) {
        // 第一次 set 会创建 table，插入 key 并返回 None（之前没值）
        let v = store.set("t1", "hello".into(), "world".into());
        assert!(v.unwrap().is_none());
        // 再次 set 同样的 key 会更新，并返回之前的值
        let v1 = store.set("t1", "hello".into(), "world1".into());
        assert_eq!(v1, Ok(Some("world".into())));

        // get 存在的 key 会得到最新的值
        let v = store.get("t1", "hello");
        assert_eq!(v, Ok(Some("world1".into())));

        // get 不存在的 key 或者 table 会得到 None
        assert_eq!(Ok(None), store.get("t1", "hello1"));
        assert!(store.get("t2", "hello1").unwrap().is_none());

        // contains 纯在的 key 返回 true，否则 false
        assert_eq!(store.contains("t1", "hello"), Ok(true));
        assert_eq!(store.contains("t1", "hello1"), Ok(false));
        assert_eq!(store.contains("t2", "hello"), Ok(false));

        // del 存在的 key 返回之前的值
        let v = store.del("t1", "hello");
        assert_eq!(v, Ok(Some("world1".into())));

        // del 不存在的 key 或 table 返回 None
        assert_eq!(Ok(None), store.del("t1", "hello1"));
        assert_eq!(Ok(None), store.del("t2", "hello"));
```

```
    }

    fn test_get_all(store: impl Storage) {
        store.set("t2", "k1".into(), "v1".into()).unwrap();
        store.set("t2", "k2".into(), "v2".into()).unwrap();
        let mut data = store.get_all("t2").unwrap();
        data.sort_by(|a, b| a.partial_cmp(b).unwrap());
        assert_eq!(
            data,
            vec![
                Kvpair::new("k1", "v1".into()),
                Kvpair::new("k2", "v2".into())
            ]
        )
    }

    fn test_get_iter(store: impl Storage) {
        store.set("t2", "k1".into(), "v1".into()).unwrap();
        store.set("t2", "k2".into(), "v2".into()).unwrap();
        let mut data: Vec<_> = store.get_iter("t2").unwrap().collect();
        data.sort_by(|a, b| a.partial_cmp(b).unwrap());
        assert_eq!(
            data,
            vec![
                Kvpair::new("k1", "v1".into()),
                Kvpair::new("k2", "v2".into())
            ]
        )
    }
}
```

这种在写实现之前写单元测试，是标准的 TDD（Test-Driven Development）方式。

我个人不是 TDD 的狂热粉丝，**但会在构建完 trait 后，为这个 trait 撰写测试代码，因为写测试代码是个很好的验证接口是否好用的时机**。毕竟我们不希望实现 trait 之后，才发现 trait 的定义有瑕疵，需要修改，这个时候改动的代价就比较大了。

所以，当 trait 推敲完毕，就可以开始写使用 trait 的测试代码了。在使用过程中仔细感受，如果写测试用例时用得不舒服，或者为了使用它需要做很多繁琐的操作，那么可以重新审视 trait 的设计。

你如果仔细看单元测试的代码，就会发现我始终秉持**测试 trait 接口的思想**。尽管在测试中需要一个实际的数据结构进行 trait 方法的测试，但核心的测试代码都用的泛型函数，让这些代码只跟 trait 相关。

这样，一来可以避免某个具体 trait 实现的干扰，二来在之后想加入更多 trait 实现时，可以共享测试代码。比如未来想支持 DiskTable，那么只消加几个测试例，调用已有的泛型函数即可。

好，搞定测试，确认 trait 设计没有什么问题之后，我们来写具体实现。可以创建 src/storage/memory.rs 来构建 MemTable：

复制代码

```rust
use crate::{KvError, Kvpair, Storage, Value};
use dashmap::{mapref::one::Ref, DashMap};

/// 使用 DashMap 构建的 MemTable，实现了 Storage trait
#[derive(Clone, Debug, Default)]
pub struct MemTable {
    tables: DashMap<String, DashMap<String, Value>>,
}

impl MemTable {
    /// 创建一个缺省的 MemTable
    pub fn new() -> Self {
        Self::default()
    }

    /// 如果名为 name 的 hash table 不存在，则创建，否则返回
    fn get_or_create_table(&self, name: &str) -> Ref<String, DashMap<String, V
        match self.tables.get(name) {
            Some(table) => table,
            None => {
                let entry = self.tables.entry(name.into()).or_default();
                entry.downgrade()
            }
        }
    }
}

impl Storage for MemTable {
    fn get(&self, table: &str, key: &str) -> Result<Option<Value>, KvError> {
        let table = self.get_or_create_table(table);
        Ok(table.get(key).map(|v| v.value().clone()))
    }

    fn set(&self, table: &str, key: String, value: Value) -> Result<Option<Val
        let table = self.get_or_create_table(table);
        Ok(table.insert(key, value))
    }

    fn contains(&self, table: &str, key: &str) -> Result<bool, KvError> {
        let table = self.get_or_create_table(table);
```

```
        Ok(table.contains_key(key))
    }

    fn del(&self, table: &str, key: &str) -> Result<Option<Value>, KvError> {
        let table = self.get_or_create_table(table);
        Ok(table.remove(key).map(|(_k, v)| v))
    }

    fn get_all(&self, table: &str) -> Result<Vec<Kvpair>, KvError> {
        let table = self.get_or_create_table(table);
        Ok(table
            .iter()
            .map(|v| Kvpair::new(v.key(), v.value().clone()))
            .collect())
    }

    fn get_iter(&self, _table: &str) -> Result<Box<dyn Iterator<Item = Kvpair>
        todo!()
    }
}
```

除了 get_iter() 外，这个实现代码非常简单，相信你看一下 dashmap 的文档，也能很快写出来。get_iter() 写起来稍微有些难度，我们先放下不表，会在下一篇 KV server 讲。如果你对此感兴趣，想挑战一下，欢迎尝试。

实现完成之后，我们可以测试它是否符合预期。注意现在 src/storage/memory.rs 还没有被添加，所以 cargo 并不会编译它。要在 src/storage/mod.rs 开头添加代码：

复制代码

```
mod memory;
pub use memory::MemTable;
```

这样代码就可以编译通过了。因为还没有实现 get_iter 方法，所以这个测试需要被注释掉：

复制代码

```
// #[test]
// fn memtable_iter_should_work() {
//     let store = MemTable::new();
//     test_get_iter(store);
// }
```

如果你运行 cargo test ，可以看到测试都通过了：

```
> cargo test
   Compiling kv v0.1.0 (/Users/tchen/projects/mycode/rust/geek-time-rust-resou
    Finished test [unoptimized + debuginfo] target(s) in 1.95s
     Running unittests (/Users/tchen/.target/debug/deps/kv-8d746b0f387a5271)

running 2 tests
test storage::tests::memtable_basic_interface_should_work ... ok
test storage::tests::memtable_get_all_should_work ... ok

test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; fi

   Doc-tests kv

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; fi
```

实现并验证 CommandService trait

Storage trait 我们就算基本验证通过了，现在再来验证 CommandService。

我们创建 src/service 目录，以及 src/service/mod.rs 和 src/service/command_service.rs 文件，并在 src/service/mod.rs 写入：

```
use crate::*;

mod command_service;

/// 对 Command 的处理的抽象
pub trait CommandService {
    /// 处理 Command，返回 Response
    fn execute(self, store: &impl Storage) -> CommandResponse;
}
```

不要忘记在 src/lib.rs 中加入 service：

```
mod error;
```

```
mod pb;
mod service;
mod storage;

pub use error::KvError;
pub use pb::abi::*;
pub use service::*;
pub use storage::*;
```

然后，在 src/service/command_service.rs 中，我们可以先写一些测试。为了简单起见，就列 HSET、HGET、HGETALL 三个命令：

复制代码

```
use crate::*;

#[cfg(test)]
mod tests {
    use super::*;
    use crate::command_request::RequestData;

    #[test]
    fn hset_should_work() {
        let store = MemTable::new();
        let cmd = CommandRequest::new_hset("t1", "hello", "world".into());
        let res = dispatch(cmd.clone(), &store);
        assert_res_ok(res, &[Value::default()], &[]);

        let res = dispatch(cmd, &store);
        assert_res_ok(res, &["world".into()], &[]);
    }

    #[test]
    fn hget_should_work() {
        let store = MemTable::new();
        let cmd = CommandRequest::new_hset("score", "u1", 10.into());
        dispatch(cmd, &store);
        let cmd = CommandRequest::new_hget("score", "u1");
        let res = dispatch(cmd, &store);
        assert_res_ok(res, &[10.into()], &[]);
    }

    #[test]
    fn hget_with_non_exist_key_should_return_404() {
        let store = MemTable::new();
        let cmd = CommandRequest::new_hget("score", "u1");
        let res = dispatch(cmd, &store);
        assert_res_error(res, 404, "Not found");
    }
```

```rust
    #[test]
    fn hgetall_should_work() {
        let store = MemTable::new();
        let cmds = vec![
            CommandRequest::new_hset("score", "u1", 10.into()),
            CommandRequest::new_hset("score", "u2", 8.into()),
            CommandRequest::new_hset("score", "u3", 11.into()),
            CommandRequest::new_hset("score", "u1", 6.into()),
        ];
        for cmd in cmds {
            dispatch(cmd, &store);
        }

        let cmd = CommandRequest::new_hgetall("score");
        let res = dispatch(cmd, &store);
        let pairs = &[
            Kvpair::new("u1", 6.into()),
            Kvpair::new("u2", 8.into()),
            Kvpair::new("u3", 11.into()),
        ];
        assert_res_ok(res, &[], pairs);
    }

    // 从 Request 中得到 Response，目前处理 HGET/HGETALL/HSET
    fn dispatch(cmd: CommandRequest, store: &impl Storage) -> CommandResponse
        match cmd.request_data.unwrap() {
            RequestData::Hget(v) => v.execute(store),
            RequestData::Hgetall(v) => v.execute(store),
            RequestData::Hset(v) => v.execute(store),
            _ => todo!(),
        }
    }

    // 测试成功返回的结果
    fn assert_res_ok(mut res: CommandResponse, values: &[Value], pairs: &[Kvpa
        res.pairs.sort_by(|a, b| a.partial_cmp(b).unwrap());
        assert_eq!(res.status, 200);
        assert_eq!(res.message, "");
        assert_eq!(res.values, values);
        assert_eq!(res.pairs, pairs);
    }

    // 测试失败返回的结果
    fn assert_res_error(res: CommandResponse, code: u32, msg: &str) {
        assert_eq!(res.status, code);
        assert!(res.message.contains(msg));
        assert_eq!(res.values, &[]);
        assert_eq!(res.pairs, &[]);
    }
}
```

这些测试的作用就是验证产品需求，比如：

HSET 成功返回上一次的值（这和 Redis 略有不同，Redis 返回表示多少 key 受影响的一个整数）

HGET 返回 Value

HGETALL 返回一组无序的 Kvpair

目前这些测试是无法编译通过的，因为里面使用了一些未定义的方法，比如 10.into()：想把整数 10 转换成一个 Value、CommandRequest::new_hgetall（“score”）：想生成一个 HGETALL 命令。

为什么要这么写？因为如果是 CommandService 接口的使用者，自然希望使用这个接口的时候，调用的整体感觉非常简单明了。

如果接口期待一个 Value，但在上下文中拿到的是 10、“hello” 这样的值，那我们作为设计者就要考虑为 Value 实现 From<T>，这样调用的时候最方便。同样的，对于生成 CommandRequest 这个数据结构，也可以添加一些辅助函数，来让调用更清晰。

到现在为止我们写了两轮测试了，相信你对测试代码的作用有大概理解。我们来总结一下：

1. 验证并帮助接口迭代
2. 验证产品需求
3. 通过使用核心逻辑，帮助我们更好地思考外围逻辑并反推其实现

前两点是最基本的，也是很多人对 TDD 的理解，其实还有更重要的也就是第三点。除了前面的辅助函数外，我们在测试代码中还看到了 dispatch 函数，它目前用来辅助测试。**但紧接着你会发现，这样的辅助函数，可以合并到核心代码中。这才是“测试驱动开发”的实质。**

好，根据测试，我们需要在 src/pb/mod.rs 中添加相关的外围逻辑，首先是 CommandRequest 的一些方法，之前写了 new_hset，现在再加入 new_hget 和 new_hgetall：

复制代码

```
impl CommandRequest {
    /// 创建 HGET 命令
    pub fn new_hget(table: impl Into<String>, key: impl Into<String>) -> Self
        Self {
            request_data: Some(RequestData::Hget(Hget {
                table: table.into(),
                key: key.into(),
            })),
        }
    }

    /// 创建 HGETALL 命令
    pub fn new_hgetall(table: impl Into<String>) -> Self {
        Self {
            request_data: Some(RequestData::Hgetall(Hgetall {
                table: table.into(),
            })),
        }
    }

    /// 创建 HSET 命令
    pub fn new_hset(table: impl Into<String>, key: impl Into<String>, value: V
        Self {
            request_data: Some(RequestData::Hset(Hset {
                table: table.into(),
                pair: Some(Kvpair::new(key, value)),
            })),
        }
    }
}
```

然后写对 Value 的 From<i64> 的实现：

复制代码

```
/// 从 i64转换成 Value
impl From<i64> for Value {
    fn from(i: i64) -> Self {
        Self {
            value: Some(value::Value::Integer(i)),
        }
    }
}
```

测试代码目前就可以编译通过了，然而测试显然会失败，因为还没有做具体的实现。我们在 src/service/command_service.rs 下添加 trait 的实现代码：

复制代码

```
impl CommandService for Hget {
    fn execute(self, store: &impl Storage) -> CommandResponse {
        match store.get(&self.table, &self.key) {
            Ok(Some(v)) => v.into(),
            Ok(None) => KvError::NotFound(self.table, self.key).into(),
            Err(e) => e.into(),
        }
    }
}

impl CommandService for Hgetall {
    fn execute(self, store: &impl Storage) -> CommandResponse {
        match store.get_all(&self.table) {
            Ok(v) => v.into(),
            Err(e) => e.into(),
        }
    }
}

impl CommandService for Hset {
    fn execute(self, store: &impl Storage) -> CommandResponse {
        match self.pair {
            Some(v) => match store.set(&self.table, v.key, v.value.unwrap_or_d
                Ok(Some(v)) => v.into(),
                Ok(None) => Value::default().into(),
                Err(e) => e.into(),
            },
            None => Value::default().into(),
        }
    }
}
```

这自然会引发更多的编译错误，因为我们很多地方都是用了 into() 方法，却没有实现相应的转换，比如，Value 到 CommandResponse 的转换、KvError 到 CommandResponse 的转换、Vec<Kvpair> 到 CommandResponse 的转换等等。

所以在 src/pb/mod.rs 里继续补上相应的外围逻辑：

复制代码

```
/// 从 Value 转换成 CommandResponse
```

```
impl From<Value> for CommandResponse {
    fn from(v: Value) -> Self {
        Self {
            status: StatusCode::OK.as_u16() as _,
            values: vec![v],
            ..Default::default()
        }
    }
}

/// 从 Vec<Kvpair> 转换成 CommandResponse
impl From<Vec<Kvpair>> for CommandResponse {
    fn from(v: Vec<Kvpair>) -> Self {
        Self {
            status: StatusCode::OK.as_u16() as _,
            pairs: v,
            ..Default::default()
        }
    }
}

/// 从 KvError 转换成 CommandResponse
impl From<KvError> for CommandResponse {
    fn from(e: KvError) -> Self {
        let mut result = Self {
            status: StatusCode::INTERNAL_SERVER_ERROR.as_u16() as _,
            message: e.to_string(),
            values: vec![],
            pairs: vec![],
        };

        match e {
            KvError::NotFound(_, _) => result.status = StatusCode::NOT_FOUND.a
            KvError::InvalidCommand(_) => result.status = StatusCode::BAD_REQU
            _ => {}
        }

        result
    }
}
```

从前面写接口到这里具体实现，不知道你是否感受到了这样一种模式：在 Rust 下，**但凡出现两个数据结构 v1 到 v2 的转换，你都可以先以 v1.into() 来表示这个逻辑，继续往下写代码，之后再去补 From<T> 的实现**。如果 v1 和 v2 都不是你定义的数据结构，那么你需要把其中之一用 struct 包装一下，来绕过（第 14 讲）之前提到的孤儿规则。

你学完这节课可以再去回顾一下第 6 讲，仔细思考一下当时说的“绝大多数处理逻辑都是把数据从一个接口转换成另一个接口”。

现在代码应该可以编译通过并测试通过了，你可以 cargo test 测试一下。

最后的拼图：Service 结构的实现

好，所有的接口，包括客户端 / 服务器的协议接口、Storage trait 和 CommandService trait 都验证好了，接下来就是考虑如何用一个数据结构把所有这些东西串联起来。

依旧从使用者的角度来看如何调用它。为此，我们在 src/service/mod.rs 里添加如下的测试代码：

复制代码

```
#[cfg(test)]
mod tests {
    use super::*;
    use crate::{MemTable, Value};

    #[test]
    fn service_should_works() {
        // 我们需要一个 service 结构至少包含 Storage
        let service = Service::new(MemTable::default());

        // service 可以运行在多线程环境下，它的 clone 应该是轻量级的
        let cloned = service.clone();

        // 创建一个线程，在 table t1 中写入 k1, v1
        let handle = thread::spawn(move || {
            let res = service.execute(CommandRequest::new_hset("t1", "k1", "v1
            assert_res_ok(res, &[Value::default()], &[]);
        });
        handle.join().unwrap();

        // 在当前线程下读取 table t1 的 k1，应该返回 v1
        let res = service.execute(CommandRequest::new_hget("t1", "k1"));
        assert_res_ok(res, &["v1".into()], &[]);
    }
}

#[cfg(test)]
use crate::{Kvpair, Value};

// 测试成功返回的结果
#[cfg(test)]
```

```
pub fn assert_res_ok(mut res: CommandResponse, values: &[Value], pairs: &[Kvpa
    res.pairs.sort_by(|a, b| a.partial_cmp(b).unwrap());
    assert_eq!(res.status, 200);
    assert_eq!(res.message, "");
    assert_eq!(res.values, values);
    assert_eq!(res.pairs, pairs);
}

// 测试失败返回的结果
#[cfg(test)]
pub fn assert_res_error(res: CommandResponse, code: u32, msg: &str) {
    assert_eq!(res.status, code);
    assert!(res.message.contains(msg));
    assert_eq!(res.values, &[]);
    assert_eq!(res.pairs, &[]);
}
```

注意，这里的 assert_res_ok() 和 assert_res_error() 是从 src/service/command_service.rs 中挪过来的。**在开发的过程中，不光产品代码需要不断重构，测试代码也需要重构来贯彻 DRY 思想。**

我见过很多生产环境的代码，产品功能部分还说得过去，但测试代码像是个粪坑，经年累月地 copy/paste 使其臭气熏天，每个开发者在添加新功能的时候，都掩着鼻子往里扔一坨走人，使得维护难度越来越高，每次需求变动，都涉及一大坨测试代码的变动，这样非常不好。

测试代码的质量也要和产品代码的质量同等要求。好的开发者写的测试代码的可读性也是非常强的。你可以对比上面写的三段测试代码多多感受。

在撰写测试的时候，我们要特别注意：**测试代码要围绕着系统稳定的部分，也就是接口，来测试，而尽可能少地测试实现。**这是我对这么多年工作中血淋淋的教训的深刻总结。

因为产品代码和测试代码，两者总需要一个是相对稳定的，既然产品代码会不断地根据需求变动，测试代码就必然需要稳定一些。

那什么样的测试代码是稳定的？测试接口的代码是稳定的。只要接口不变，无论具体实现如何变化，哪怕今天引入一个新的算法，明天重写实现，测试代码依旧能够凛然不动，做好产品质量的看门狗。

好，我们回来写代码。在这段测试中，已经敲定了 Service 这个数据结构的使用蓝图，它可以跨线程，可以调用 execute 来执行某个 CommandRequest 命令，返回 CommandResponse。

根据这些想法，在 src/service/mod.rs 里添加 Service 的声明和实现：

复制代码

```rust
/// Service 数据结构
pub struct Service<Store = MemTable> {
    inner: Arc<ServiceInner<Store>>,
}

impl<Store> Clone for Service<Store> {
    fn clone(&self) -> Self {
        Self {
      inner: Arc::clone(&self.inner),
        }
    }
}

/// Service 内部数据结构
pub struct ServiceInner<Store> {
    store: Store,
}

impl<Store: Storage> Service<Store> {
    pub fn new(store: Store) -> Self {
        Self {
            inner: Arc::new(ServiceInner { store }),
        }
    }

    pub fn execute(&self, cmd: CommandRequest) -> CommandResponse {
        debug!("Got request: {:?}", cmd);
        // TODO: 发送 on_received 事件
        let res = dispatch(cmd, &self.inner.store);
        debug!("Executed response: {:?}", res);
        // TODO: 发送 on_executed 事件

        res
    }
}

// 从 Request 中得到 Response，目前处理 HGET/HGETALL/HSET
pub fn dispatch(cmd: CommandRequest, store: &impl Storage) -> CommandResponse
    match cmd.request_data {
        Some(RequestData::Hget(param)) => param.execute(store),
        Some(RequestData::Hgetall(param)) => param.execute(store),
```

```
        Some(RequestData::Hset(param)) => param.execute(store),
        None => KvError::InvalidCommand("Request has no data".into()).into(),
        _ => KvError::Internal("Not implemented".into()).into(),
    }
}
```

这段代码有几个地方值得注意：

1. 首先 Service 结构内部有一个 ServiceInner 存放实际的数据结构，Service 只是用 Arc 包裹了 ServiceInner。这也是 Rust 的一个惯例，把需要在多线程下 clone 的主体和其内部结构分开，这样代码逻辑更加清晰。
2. execute() 方法目前就是调用了 dispatch，但它未来潜在可以做一些事件分发。这样处理体现了 SRP（Single Responsibility Principle）原则。
3. dispatch 其实就是把测试代码的 dispatch 逻辑移动过来改动了一下。

再一次，我们重构了测试代码，把它的辅助函数变成了产品代码的一部分。现在，你可以运行 `cargo test` 测试一下，如果代码无法编译，可能是缺一些 use 代码，比如：

复制代码

```
use crate::{
    command_request::RequestData, CommandRequest, CommandResponse, KvError, Me
};
use std::sync::Arc;
use tracing::debug;
```

新的 server

现在处理逻辑已经都完成了，可以写个新的 example 测试服务器代码。

把之前的 examples/dummy_server.rs 复制一份，成为 examples/server.rs，然后引入 Service，主要的改动就三句：

复制代码

```
// main 函数开头，初始化 service
let service: Service = Service::new(MemTable::new());
// tokio::spawn 之前，复制一份 service
let svc = service.clone();
// while loop 中，使用 svc 来执行 cmd
```

```
6 let res = svc.execute(cmd);
```

你可以试着自己修改。完整的代码如下：

复制代码

```
1  use anyhow::Result;
2  use async_prost::AsyncProstStream;
3  use futures::prelude::*;
4  use kv::{CommandRequest, CommandResponse, MemTable, Service};
5  use tokio::net::TcpListener;
6  use tracing::info;
7
8  #[tokio::main]
9  async fn main() -> Result<()> {
10     tracing_subscriber::fmt::init();
11     let service: Service = Service::new(MemTable::new());
12     let addr = "127.0.0.1:9527";
13     let listener = TcpListener::bind(addr).await?;
14     info!("Start listening on {}", addr);
15     loop {
16         let (stream, addr) = listener.accept().await?;
17         info!("Client {:?} connected", addr);
18         let svc = service.clone();
19         tokio::spawn(async move {
20             let mut stream =
21                 AsyncProstStream::<_, CommandRequest, CommandResponse, _>::fro
22             while let Some(Ok(cmd)) = stream.next().await {
23                 let res = svc.execute(cmd);
24                 stream.send(res).await.unwrap();
25             }
26             info!("Client {:?} disconnected", addr);
27         });
28     }
29 }
```

完成之后，打开一个命令行窗口，运行：`RUST_LOG=info cargo run --example server --quiet`，然后在另一个命令行窗口，运行：`RUST_LOG=info cargo run --example client --quiet`。此时，服务器和客户端都收到了彼此的请求和响应，并且处理正常。

我们的 KV server 第一版的基本功能就完工了！当然，目前还只处理了 3 个命令，剩下 6 个需要你自己完成。

小结

KV server 并不是一个很难的项目，但想要把它写好，并不简单。如果你跟着讲解一步步走下来，可以感受到一个有潜在生产环境质量的 Rust 项目应该如何开发。在这上下两讲内容中，有两点我们一定要认真领会。

第一点，你要对需求有一个清晰的把握，找出其中不稳定的部分（variant）和比较稳定的部分（invariant）。在 KV server 中，不稳定的部分是，对各种新的命令的支持，以及对不同的 storage 的支持。**所以需要构建接口来消弭不稳定的因素，让不稳定的部分可以用一种稳定的方式来管理。**

第二点，代码和测试可以围绕着接口螺旋前进，使用 TDD 可以帮助我们进行这种螺旋式的迭代。**在一个设计良好的系统中：接口是稳定的，测试接口的代码是稳定的，实现可以是不稳定的。**在迭代开发的过程中，我们要不断地重构，让测试代码和产品代码都往最优的方向发展。

纵观我们写的 KV server，包括测试在内，你很难发现有函数或者方法超过 50 行，代码可读性非常强，几乎不需要注释，就可以理解。另外因为都是用接口做的交互，未来维护和添加新的功能，也基本上满足 OCP 原则，除了 dispatch 函数需要很小的修改外，其它新的代码都是在实现一些接口而已。

相信你能初步感受到在 Rust 下撰写代码的最佳实践。如果你之前用其他语言，已经采用了类似的最佳实践，那么可以感受一下同样的实践在 Rust 下使用的那种优雅；如果你之前由于种种原因，写的是类似之前意大利面条似的代码，那在开发 Rust 程序时，你可以试着接纳这种更优雅的开发方式。

毕竟，现在我们手中有了更先进的武器，就可以用更先进的打法。

思考题

1. 为剩下 6 个命令 HMGET、HMSET、HDEL、HMDEL、HEXIST、HMEXIST 构建测试，并实现它们。在测试和实现过程中，你也许需要添加更多的 From<T> 的实现。
2. 如果有余力，可以试着实现 MemTable 的 get_iter() 方法（后续的 KV Store 实现会讲）。

延伸思考

虽然我们的 KV server 使用了 concurrent hashmap 来处理并发，但这并不一定是最好的选择。

我们也可以创建一个线程池，每个线程有自己的 HashMap。当 HGET/HSET 等命令来临时，可以对 key 做个哈希，然后分派到 “拥有” 那个 key 的线程，这样，可以避免在处理的时候加锁，提高系统的吞吐。你可以想想如果用这种方式处理，该怎么做。

恭喜你完成了学习的第 22 次打卡。如果你觉得有收获，也欢迎你分享给身边的朋友，邀他一起讨论。我们下一讲期中测试见～

分享给需要的人，Ta订阅后你可得 20 元现金奖励

生成海报并分享

赞 6 提建议

© 版权归极客邦科技所有，未经许可不得传播售卖。 页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

上一篇 21 | 阶段实操：构建一个简单的 KV server (1) -基本流程（上）

下一篇 加餐 | 期中测试：来写一个简单的 grep 命令行



精选留言 (5)

写留言

pedro
2021-10-11

得益于老师良好的抽象，我抽出了中午的时间，完成了 hdel 和 hexist 两个命令，如下：
```
running 6 tests
test service::command_service::tests::hget_with_non_exist_key_should_return_404 ... ok...
```
展开

2

newzai
2021-10-11

dev-dependencies 与 dependencies的第三方crate是如何划分的?tokio 为啥不放到 dependencies? 放到 dependencies 与 dev-dependencies 有啥区别？某些如何决策一个 crate放到哪个 dependencies？

2 1

2021-10-15

```Rust
impl Kvpair {
```

```
    /// 创建一个新的 kv pair
    fn new(key: impl Into<String>, value: impl Into<Value>) -> Self {
        Self {…
```

展开

 



终生恻隐

2021-10-13

```
// hmget
    // 创建 HGETALL 命令
    pub fn new_hmget(table: impl Into<String>, keys: Vec::<impl Into<String> + Copy
>) -> Self {
        let mut v: Vec::<::prost::alloc::string::String> = Vec::new();…
```

展开

 



pedro

2021-10-12

文中代码有一处错误：

```
let cloned = service.clone();
// 创建一个线程，在 table t1 中写入 k1, v1
let handle = thread::spawn(move || {
      let res = cloned.execute(CommandRequest::new_hset("t1", "k1", "v1".into()));…
```

展开

 